

Circuit Codex

User manual

What each tool does, what its terms mean, and where its numbers come from. Tap a tool to go to its section.

Index

PASSIVE COMPONENTS

Resistor color code	Resistors
---------------------	-----------

DIGITAL & LOGIC CIRCUITS

Karnaugh map simplification	Logic gates
I2C pull-up resistor	Timing & interfaces
UART baud rate	Timing & interfaces
Crystal load capacitance	Timing & interfaces
Oscillator stability	Timing & interfaces
PLL multiplication factor	Timing & interfaces
ADC resolution / quantization	Data conversion
DAC resolution	Data conversion
SNR estimation	Data conversion

Resistor color code

What it computes

The resistance, tolerance and, on 6-band parts, temperature coefficient that a resistor's colour bands stand for — and the other way round: type a value and it shows the bands. It also says whether the value is a standard E-series value and, if not, the nearest one.

Source

IEC 60062:2016, *Marking codes for resistors and capacitors*. Its table gives each colour a digit, a multiplier, a tolerance and a temperature coefficient.

What the bands mean

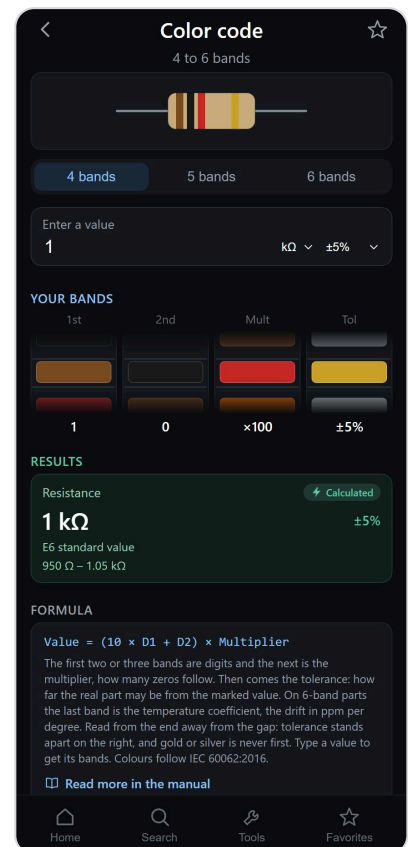
A resistor is too small to print a number on, so the value is painted as rings of colour, one digit per ring. Each colour stands for a digit from 0 to 9: black 0, brown 1, red 2, orange 3, yellow 4, green 5, blue 6, violet 7, grey 8, white 9.

- **Digit bands** — the first two (4-band parts) or three (5- and 6-band parts) give the significant figures.
- **Multiplier** — the next band says how many zeros follow, as a power of ten: red is $\times 100$, orange $\times 1000$. Gold ($\times 0.1$) and silver ($\times 0.01$) make values below 10 Ω .
- **Tolerance** — how far the actual resistance may be from the marked value: gold $\pm 5\%$ means a 1 k Ω part measures somewhere from 950 Ω to 1050 Ω . Leaving the band off means $\pm 20\%$.
- **Temperature coefficient** (6-band only) — how much the resistance drifts per degree of temperature change, in parts per million per kelvin: brown is 100 ppm/K, so 0.01% per degree.

Brown–black–red–gold is therefore 1, 0, $\times 100$, $\pm 5\%$: 1 k Ω $\pm 5\%$.

Which end to start from. The tolerance band usually stands apart from the others, on the right; the digits are bunched on the left. Gold and silver are never digits, so a part with gold at one end is read from the other.

Standard values. Resistors are made only in preferred values, the E-series, spaced so that neighbouring values just overlap within their tolerance. The tool reports the coarsest series the value belongs to — 4.7 k Ω is an E6 value even when bought at 1% — and when the value is in none, the nearest one in the series its tolerance implies.



How the value is worked out

4 bands: $\text{Value} = (10 \times D1 + D2) \times \text{Multiplier}$
5, 6: $\text{Value} = (100 \times D1 + 10 \times D2 + D3) \times \text{Multiplier}$

Typing a value runs it backwards: the value is split into as many significant digits as the band count allows, rounded, and the remaining power of ten picks the multiplier colour. The smallest value a code can show is 0.1 Ω on 4 bands (brown–black–silver) and 1 Ω on 5 or 6; the largest is 99 G Ω and 999 G Ω . Outside that there is no band for it, and the tool says so.

Assumptions and limits

- **The 2016 tolerance colours.** IEC 60062:2016 made orange $\pm 0.05\%$, yellow $\pm 0.02\%$ and grey $\pm 0.01\%$. Before that edition some manufacturers used grey for a non-standard $\pm 0.05\%$, so a grey tolerance band on an old part may mean that.
- **Colours are only as good as the light.** Red, orange and brown are easily confused on a real part, and so are violet and grey. When in doubt, measure.
- **Zero-ohm links** are marked with a single black band and are not a colour code as such.

What it deliberately does not do

- **Capacitor and inductor colour codes** have their own tools, since the same colours mean different units and tolerances there.
- **SMD resistor codes** (the three- and four-digit and EIA-96 markings) are the SMD code tool.

How it was checked

Against the IEC 60062:2016 table: digits, multipliers, tolerances and temperature coefficients for every colour. This check found the app had grey as $\pm 0.05\%$, the pre-2016 non-standard value, and no tolerance for orange or yellow; all three now follow the standard.

Brown–black–red–gold reads 1 k Ω $\pm 5\%$, 950 Ω to 1.05 k Ω , E6. At grey's $\pm 0.01\%$ the range reads 999.9 Ω to 1.0001 k Ω — at four figures it had printed as "1 k Ω ", which the range now shows to enough figures to see. Typing 4.7 k gives yellow–violet–red; 0.22 Ω gives red–red–silver.

Karnaugh map simplification

What it computes

Takes a truth table for 2, 3 or 4 variables — each cell either 0, 1 or a don't-care — and returns the minimum sum-of-products expression, with each product term drawn as a group on the map in its own colour.

Source

The map is Karnaugh's (1953), itself a reworking of the Veitch diagram. The minimisation is Quine–McCluskey, which is an exact algorithm rather than a heuristic; the covering step is solved by exhaustive search, which is affordable at four variables and would not be at twenty.

Why the formulas are these

The whole method rests on the ordering along the edges: **00, 01, 11, 10**, not 00, 01, 10, 11. That is Gray code, and it means any two side-by-side cells differ in exactly one variable. When two adjacent cells are both 1, that variable appears once true and once complemented, so it cancels:

$$A \cdot \bar{B} \cdot C + A \cdot B \cdot C = A \cdot C \cdot (\bar{B} + B) = A \cdot C$$

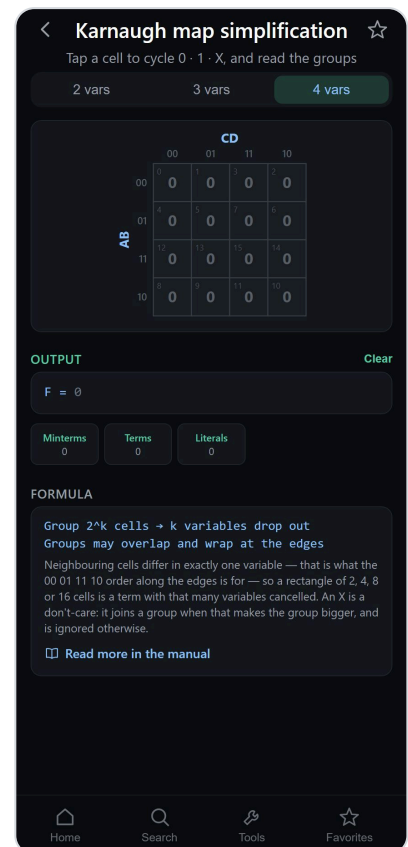
Extend it and a rectangle of 2^k cells cancels k variables. A pair drops one, a block of four drops two, eight drops three. That is the entire trick — the map is just an arrangement that puts algebraically adjacent terms physically next to each other.

The edges wrap. The first and last columns also differ in one variable (00 and 10 differ only in the middle bit), and so do the first and last rows. The map is really a torus. That is why the four corners of a four-variable map form one group of four, and why a group can run off one edge and continue on the other. A torus rectangle has no single box on a flat sheet, so the tool draws wrapping groups as two or four pieces in the same colour.

Quine–McCluskey. Terms are combined pairwise wherever they differ in one bit, the shared bit becoming a dash; repeat until nothing more combines. What never combined is a **prime implicant** — a group that cannot be made larger. Then a cover must be chosen: a minterm covered by exactly one prime implicant forces that implicant in (it is *essential*), and the rest is a set-cover problem solved here by branch and bound.

The cover is ordered **fewest terms first, then fewest literals**. The second key matters: several covers can tie on term count while differing in how many variables survive, and the answer a textbook prints is the one with fewer letters.

Don't-cares are included when forming groups but never required to be covered. That is where most of the saving usually comes from: an X adjacent to a group lets the group double in size, dropping another variable



for free.

Parity is what the map cannot see. A checkerboard of ones has no two adjacent cells, so every group is a single cell and the sum of products comes out at full length — twelve of twelve literals on a three-variable map, nothing saved at all. That function is exclusive-OR:

$$A \oplus B \oplus C = 1 \text{ whenever an odd number of } A, B, C \text{ are } 1$$

Two XOR gates implement it against four ANDs and an OR for the sum of products. The map cannot find this because XOR is not a sum of products in any small form — its minimal SOP genuinely is the long one. The tool therefore tests, separately from the map, whether the function is the parity of any subset of its variables (or the complement of one) and prints that form when it is.

Assumptions and limits

- Two to four variables. Five and six variable maps exist, drawn as stacked pairs, but they stop being readable and the whole point of the map is that you can see it.
- The result is a minimum, not *the* minimum: several different expressions can tie on both term and literal count, and the tool shows one of them.
- Literal count is a proxy for cost, not a gate count. It ignores that inverters may be free if a signal is already available complemented, and that fan-in is limited in real logic families.
- No account of hazards. A minimal cover can contain a static hazard — a glitch when two inputs change at once — which is removed by adding a redundant group that the minimiser will never choose because it costs a term.

What it deliberately does not do

- **Product of sums.** The minimal POS is the same procedure applied to the zeros, then inverted. It would double the interface for a form that is asked for far less often.
- **More than four variables,** as above.
- **Hazard-free covers.** Adding the redundant terms is a different objective from minimisation and would make the answers disagree with what a textbook shows.

How it was checked

A second minimiser was written independently for the purpose: enumerate every possible cube, discard those covering a zero, and find the smallest cover by iterative deepening. The two were compared over 3500 random functions at 2, 3 and 4 variables, and 12000 more were checked only for the expression reproducing the truth table at every care position.

That comparison earned its keep. Term counts matched from the start, but 34 of the first 3400 came back with more literals than the reference — the cover search was returning whichever minimum-size cover it reached first, with no tie-break on literals. A correct minimum by term count, and still not the printed answer.

The parity detector was checked exhaustively over all 256 three-variable functions: exactly eight are parity forms (three pairs and one triple, each with its complement), and each one's exclusive-OR form reproduces its truth table.

I2C pull-up resistor

What it computes

The window of pull-up resistances that work on an I²C bus, given the supply, the bus capacitance and the speed mode — and a suggested value inside it, snapped to a standard E-series.

Source

The I²C specification, **NXP UM10204**. Every constant here is from it: the 0.4 V output-low level, the 3 mA and 20 mA sink currents, the rise-time budgets, and the bus capacitance ceilings.

Why the formulas are these

I²C lines are **open-drain**. Nothing on the bus drives a line high; devices can only pull it low, and the pull-up resistor is what brings it back up. That single fact produces both bounds, pulling in opposite directions.

The lower bound is the pull-down. When a device asserts the line, its open-drain transistor has to hold it below the 0.4 V that counts as a low, and the current it must swallow is whatever the pull-up pushes through:

$$R_{p \min} = (V_{dd} - 0.4 \text{ V}) / I_{OL}$$

The spec requires devices to sink **3 mA** up to Fast mode, and **20 mA** in Fast mode Plus. A smaller resistor asks for more than the transistor is rated to take, and the line stops reaching a valid low.

The upper bound is the rise. Going up, the line is just an RC: the pull-up charging the bus capacitance. The spec measures rise time between **0.3 V_{dd}** and **0.7 V_{dd}**, and an RC charge reaches those fractions at

$$t(0.3 \text{ V}_{dd}) = RC \cdot \ln(1/0.7)$$

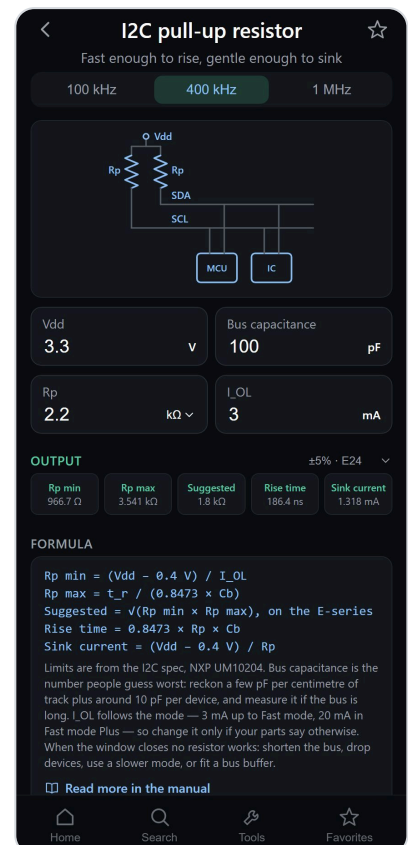
$$t(0.7 \text{ V}_{dd}) = RC \cdot \ln(1/0.3)$$

so the specified rise time is the difference:

$$t_r = RC \cdot [\ln(1/0.3) - \ln(1/0.7)] = RC \cdot \ln(7/3) = 0.8473 \cdot RC$$

Hence

$$R_{p \max} = t_r / (0.8473 \times C_b)$$



That 0.8473 is copied everywhere without explanation; it is just $\ln(7/3)$. The budget for t_r is 1000 ns at 100 kHz, 300 ns at 400 kHz and 120 ns at 1 MHz.

The suggestion is the geometric mean of the two bounds, $\sqrt{R_{p \text{ min}} \times R_{p \text{ max}}}$, snapped to the chosen E-series. The geometric mean is the centre of the window on a logarithmic scale, which is what "equal margin on both sides" means when the two ends can differ by a decade. For a 3.3 V standard-mode bus with 100 pF it lands on 3.3 k Ω — the value most designs actually fit.

When the window closes. $R_{p \text{ max}}$ falls as bus capacitance rises, while $R_{p \text{ min}}$ does not move. Past roughly 366 pF in Fast mode at 3.3 V the two cross and no resistor satisfies both: the bus cannot rise fast enough without asking the pull-down for more current than it is rated for. That is a real dead end, not a poor choice, and the ways out are a shorter bus, fewer devices, a slower mode, or an active bus buffer that isolates segments.

Assumptions and limits

- **Bus capacitance is an estimate.** Reckon on a few pF per centimetre of track plus about 10 pF per device. It is the input people are most wrong about, and the whole upper bound scales with it. Measure it on a long bus.
- The RC model treats the bus as one lumped capacitance and the pull-up as ideal. At Fast mode Plus, with long tracks, transmission-line behaviour starts to matter and the model flatters the real rise time.
- Leakage from many devices and from the pull-up itself is ignored. On a bus with dozens of devices it adds a small standing current that eats into the low-level margin.
- I_{OL} defaults to the spec value for the mode. A specific part may be rated differently; the field is editable for that reason.

What it deliberately does not do

- **Current-source or active pull-ups.** Fast mode Plus devices sometimes use a current source rather than a resistor, which removes the RC ceiling entirely. Different circuit, different arithmetic.
- **Bus buffers and repeaters.** They break a bus into segments each with their own capacitance and their own resistor; the tool sizes one segment, so run it once per segment.
- **Level shifters.** A MOSFET level shifter between two voltage domains puts a pull-up on each side, and the two interact. Not modelled.

UART baud rate

What it computes

The divisor a UART needs to reach a wanted baud rate from a given peripheral clock, the rate it actually produces once that divisor is rounded to an integer, and the resulting error — with the error shown against every standard rate on the same clock.

Source

There is no single standard for baud generation: it is per-device, and the register is called BRR, UBRR, DLL/DLM or SBRG depending on whose part it is. What is common to nearly all of them is the shape — an integer divider off a peripheral clock, with the receiver oversampling each bit — and 16x oversampling, inherited from the 8250 and 16550 and still the default almost everywhere. The error budget below is derived from the sampling behaviour, not quoted.

Why the formulas are these

A UART has no oscillator of its own. It divides whatever clock it is given:

$$\text{Divisor} = \text{Clock} / (\text{Oversample} \times \text{Baud})$$

The divider is a counter, so the value written is an **integer**. Rounding it is the whole problem:

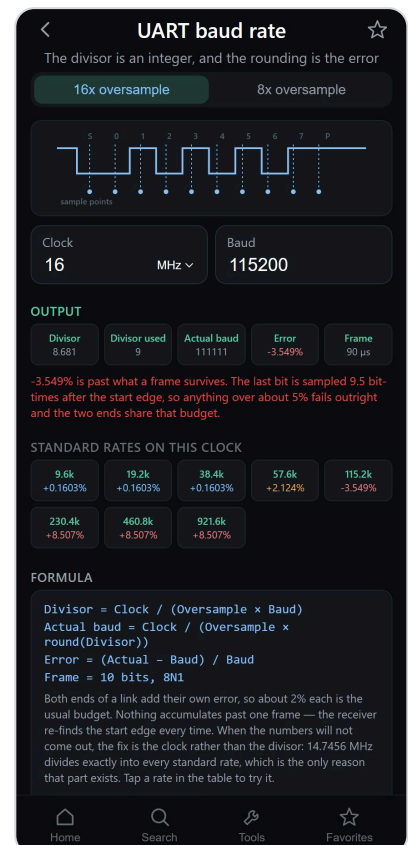
$$\text{Actual baud} = \text{Clock} / (\text{Oversample} \times \text{round}(\text{Divisor}))$$

At 16 MHz, 115200 baud and 16x oversampling the divisor comes to 8.68. Written as 9, the line runs at 111111 baud — 3.55% slow. That single fact is why 14.7456 MHz crystals exist: $14745600 / 16 = 921600$, which divides exactly into every standard rate, so the divisor is always a whole number and the error is zero. 11.0592 MHz and 18.432 MHz are the same trick at other speeds.

Why the error matters, and how much is allowed. There is no clock shared between the two ends. The receiver finds the falling edge of the start bit, starts its own counter, and samples each bit at what it believes is the middle. Take the receiver as the reference: it samples bit n at time

$$(n + 0.5) \times T_{rx}$$

while the transmitter's bit n occupies



$$[n \times T_{tx} , (n+1) \times T_{tx}]$$

Writing $r = T_{tx} / T_{rx}$, the sample lands in the right bit only while

$$n \times r < n + 0.5 < (n+1) \times r$$

Nothing resynchronises inside a frame, so the worst case is the last bit. For 8N1 that is the stop bit, $n = 9$:

$$\begin{aligned} 9r < 9.5 & \rightarrow r < 1.0556 \\ 9.5 < 10r & \rightarrow r > 0.95 \end{aligned}$$

So the two clocks may differ by about **−5.0% to +5.56%** — usually quoted as $\pm 5\%$, and slightly asymmetric because the sample point is fixed at mid-bit while the bit it must land in is what stretches. That budget covers **both** ends together, which is why 2% each is the practical rule: it leaves room for the other end plus edge jitter and finite rise time.

Note what does *not* happen: the error does not accumulate across a message. The receiver re-finds the start edge on every frame, so the drift resets ten bits at a time. A 3% error is not 3% worse each byte; it is the same 3% on each of them, which is why it either works or does not.

The drawing shows exactly this. The waveform is drawn at the transmitter's real bit width, stretched or squeezed by the error, while the sample marks stay evenly spaced where the receiver puts them. Raise the error and the marks visibly walk toward the bit edges; the ones that fall outside their own bit turn red. It is the inequality above, drawn.

Assumptions and limits

- **8N1 is assumed** — one start bit, eight data, one stop, no parity. Adding parity or a second stop bit makes the frame longer and the budget tighter, since the worst case moves to a later bit.
- **The receiver is taken as exact.** In reality both ends have error and the two add; the tool reports one side's, which is why the caution appears at 2% rather than 5%.
- **Integer divisors only.** See below.
- Oversampling is a divider stage, not noise immunity: 8x doubles the top speed from a given clock and halves the margin around each sample, so a noisy line suffers more.

What it deliberately does not do

- **Fractional dividers.** Many modern UARTs — STM32, SAM, nRF — add fractional bits below the integer divisor, which cuts the error by roughly the resolution of that fraction and would make the numbers here pessimistic for those parts. Modelling it means picking a vendor, since the fraction is 4 bits on some parts, 6 on others, and a full 16-bit accumulator on a few.
- **Auto-baud detection**, where the receiver measures a known character instead of being told the rate.
- **Anything above the physical layer:** framing errors, break detection, flow control.

How it was checked

The classic cases were recomputed by hand: 16 MHz at 115200 gives divisor 8.68, used 9, 111111 baud, -3.549%; 14.7456 MHz gives exactly 8 and 0%; 16 MHz at 9600 gives 104.17, used 104, +0.160%, which is why that part runs 9600 happily and 115200 badly. 8 MHz at 115200 gives +8.507%.

The sample-point test in the drawing was checked against the inequality above at several errors: at 0% and -3.5% every sample lands inside its bit, at +0.16% likewise, and at +8.5% the last four fail — which is what $n \times 0.9216 < n + 0.5$ predicts, the first failure being at $n = 6$.

Crystal load capacitance

What it computes

The two capacitors a Pierce oscillator needs so the crystal sees the load it was cut for, the load a given pair actually presents, and how far off frequency the difference pulls it.

Source

The series expression appears in every oscillator application note — ST AN2867, the Abracon and Epson design guides, and the NXP equivalents. The pulling expression comes from the crystal's equivalent circuit, the Butterworth–Van Dyke model: a motional arm of L1, C1 and R1 in series, all in parallel with the shunt capacitance C0 of the electrodes and holder.

Why the formulas are these

The two capacitors are in series, not in parallel. This is the step people get backwards. Each one runs from one crystal terminal to ground, so the loop the crystal drives is: out through CL1, along the ground, back through CL2. Two capacitors in series:

$$CL1 \times CL2 / (CL1 + CL2)$$

which for an equal pair is just half of one of them. Add the stray capacitance of the two tracks and the two oscillator pins — that sits directly across the same nodes, so it adds on:

$$\text{Load} = CL1 \times CL2 / (CL1 + CL2) + \text{Stray}$$

Turn it round for a symmetric pair and the capacitors come out at

$$CL1 = CL2 = 2 \times (CL \text{ spec} - \text{Stray})$$

A crystal marked 12.5 pF on a board with 3 pF of stray therefore wants **19 pF** parts, not 25 pF and not 12.5 pF. Both of those are common mistakes, and the second is the worse one.

Why the load changes the frequency at all. A crystal has two resonances. At series resonance the motional arm is purely resistive; a little above it the arm looks inductive and resonates with everything capacitive across it — C0 and whatever load the circuit adds. The parallel-resonant frequency is



$$f_p = f_s \times [1 + C_1 / (2 \times (C_0 + C_L))]$$

A crystal is cut and trimmed so that this lands on the marked frequency **for one particular CL**. Present a different load and it lands somewhere else, which is the whole reason the number is on the datasheet.

Differentiating that with respect to CL gives the sensitivity, which is what the tool reports as pullability:

$$d(\Delta f/f) / dC_L = - C_1 / (2 \times (C_0 + C_L)^2)$$

The sign is negative: **more load, lower frequency**. For a typical MHz part with $C_1 = 8 \text{ fF}$ and $C_0 = 3 \text{ pF}$ at $C_L = 12.5 \text{ pF}$ that comes to about 16.6 ppm/pF, so fitting 22 pF where 19 pF was wanted — 1.5 pF too much load — pulls it roughly 25 ppm slow. A 32.768 kHz tuning fork has a tenth the motional capacitance and pulls far less per pF, around 6 ppm/pF on the same load, which is why watch crystals are specified so tightly: there is little room to trim them back.

Where the inputs come from, and what they are called

Three of the inputs are easy to look for in the wrong place.

- **C0** — on the crystal datasheet, as **C0**, "shunt capacitance" or "static capacitance". Often given only as a maximum; that is usable.
- **C1** — the motional capacitance, **C1** on datasheets that give it, after the Butterworth–Van Dyke model. **Many do not.** When it is missing, look for a pullability figure in ppm/pF, or a "frequency versus load capacitance" curve, and type that into Pullability instead: the tool works C1 back from it. C1 is the one that stays fixed when the load changes — it is the crystal — while pullability is its slope at whatever load is chosen, so changing CL spec recomputes pullability, not C1.
- **Stray** — **never** on the crystal datasheet, because it is not the crystal. It is the capacitance of the two oscillator pins, sometimes listed in the MCU datasheet, plus the two tracks. Application notes such as ST AN2867 give 2–5 pF as the usual range; that is an order of magnitude, not a value to copy.

The load capacitors are called **CL1 and CL2** rather than C1 and C2, which is how MCU application notes label them and which keeps C1 free for its datasheet meaning. Every capacitance here only ever comes in one unit — pF for load, stray and shunt, fF for motional — so the fields show the unit fixed rather than offering a picker.

When CL says "Series". Some datasheets offer, or list, a series-resonant cut: the crystal is trimmed to its marked frequency with no load at all. A Pierce oscillator cannot present no load, so the part runs above its marking by the whole parallel-resonance offset,

$$\Delta f/f = C_1 / (2 \times (C_0 + \text{Load}))$$

which is not a trim but a few hundred ppm: 8 fF against 3 pF + 12.5 pF is +258 ppm. Type 0 in CL spec to see it. The fix is to order the part with a load capacitance, or to use it in a circuit that runs it at series resonance.

Assumptions and limits

- **Stray is an estimate and it dominates the mistakes.** Two to five picofarads is the usual range for the pins plus short tracks, but a long or guarded layout can be well outside it. If the frequency matters, measure.
- The presets are typical for each family, not for any specific part, and a wrong C1 scales the pullability proportionally.
- The pulling figure is a **slope taken at the specified load**, so it is exact for small errors and increasingly optimistic for large ones, since the true curve flattens as CL grows.
- Sizing assumes a **symmetric pair**. Asymmetric capacitors are legitimate and the tool will report the load they present, but it will not propose them.

What it deliberately does not do

- **Drive level and negative-resistance margin.** Whether the oscillator starts and whether it overdrives the crystal are the other half of the design, set by the amplifier's transconductance, the crystal's ESR and any series resistor. It is a separate calculation with separate datasheet numbers.
- **The feedback resistor**, which is inside the MCU on every part this applies to and is not drawn for that reason.
- **Temperature and ageing.** The load error here is a fixed offset; drift over temperature is the crystal's cut and belongs with the ppm tooling.

How it was checked

Recomputed by hand: 12.5 pF specified with 3 pF stray gives 19 pF capacitors and a presented load of exactly 12.5 pF, 0 ppm. Fitting 22 pF instead presents 14 pF, 1.5 pF over, and pulls -24.97 ppm against $16.65 \times 1.5 = 24.98$ by hand. Fitting 33 pF presents 19.5 pF and pulls -116.5 ppm against $16.65 \times 7 = 116.6$. The watch preset returns 6.378 ppm/pF against $2.5 \text{ fF} / (2 \times (1.5 + 12.5 \text{ pF})^2) = 6.38$ by hand.

The pullability was wrong by twelve orders of magnitude on first write: $C1 / (2(C0+CL)^2)$ is *per farad*, not dimensionless, because the numerator is first order in capacitance and the denominator is second. It needs multiplying by one picofarad to become per-pF. The hand check is what caught it.

Oscillator stability

What it computes

The total frequency error budget of a crystal oscillator — tolerance, temperature, ageing and load pulling combined — as a worst case and as a root-sum-square estimate, then expressed as hertz and as time gained or lost per day and per year.

For a single ppm figure applied to a single value, the **PPM converter** under Tools does that; this tool is the budget that produces the figure.

Source

The contributions and their names are the ones on crystal datasheets: frequency tolerance at a reference temperature, frequency stability over the operating temperature range, and ageing, usually per first year. The reference is 20 °C on some datasheets and 25 °C on others — one Petermann-Technik page quotes both for the same part — so the field is labelled Tolerance without a temperature. The 32.768 kHz temperature curve is the parabola the tuning-fork datasheets from Epson, Abracon and Micro Crystal all quote, with a coefficient of about $-0.034 \text{ ppm}/^{\circ}\text{C}^2$ (± 0.006) about a turnover near 25 °C.

Why the formulas are these

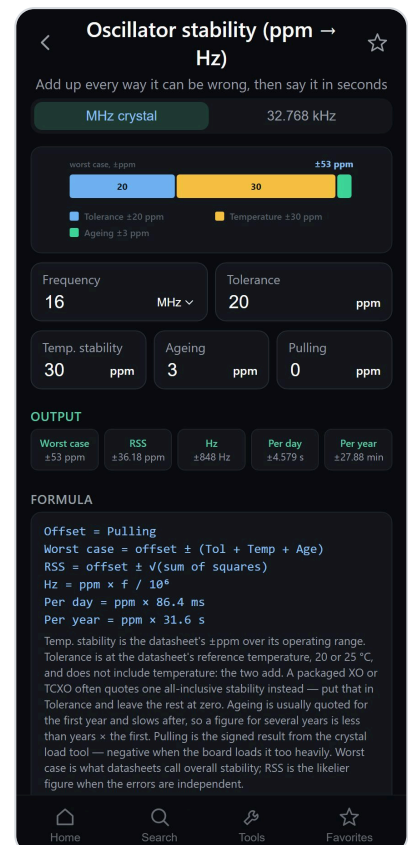
Two kinds of error, kept apart. Tolerance, temperature stability and ageing are *limits*: the part is somewhere within $\pm$ each of them, and which way it sits is not known. The tuning fork's temperature drift and the load pulling are *offsets*: they have a known sign and they move the whole window. Mixing the two kinds is how budgets end up either too pessimistic or wrong-signed, so the tool computes

$$\begin{aligned}\text{Offset} &= \text{Drift} + \text{Pulling} \\ \text{Worst case} &= \text{Offset} \pm (\text{Tolerance} + \text{Temperature} + \text{Ageing})\end{aligned}$$

and reports the extreme of that window with its sign. A window of -69 to -23 ppm is a clock that can only lose time, and writing it as ± 69 ppm would throw that away.

Worst case and RSS. The worst case adds every limit at full size in the same direction. It is what datasheets call *overall stability*, and it is the right figure to design against when failure is not an option. If the contributions are independent, all of them sitting at their extremes together is unlikely, and the root-sum-square

$$\text{RSS} = \text{Offset} \pm \sqrt{(\text{Tolerance}^2 + \text{Temperature}^2 + \text{Ageing}^2)}$$



is the likelier spread. Offsets are added directly, never squared: they are not random.

The tuning-fork parabola. A 32.768 kHz crystal is a quartz tuning fork, and its frequency falls off either side of a turnover temperature:

$$\Delta f/f = -0.034 \text{ ppm/}^{\circ}\text{C}^2 \times (T - 25 \text{ }^{\circ}\text{C})^2$$

It only ever goes down. At 0 °C that is –21 ppm, about 1.8 seconds a day; at –20 °C it is –69 ppm, six seconds a day. That is why a watch or a data logger left outdoors in winter runs slow, and why the tool asks for the temperature the fork will actually see rather than a spec. MHz crystals are AT-cut plates, whose curve is a cubic set by the cut angle; their datasheets fold it into one ±ppm figure over the range, which is what the MHz pill asks for.

Into time. A day is 86 400 s, so 1 ppm is 86.4 ms a day, and a year of 31 557 600 s makes 1 ppm about 31.6 s a year. Twenty ppm is 1.73 s a day or 10.5 minutes a year.

Assumptions and limits

- **Ageing is not linear.** It is fastest in the first months and slows roughly logarithmically, so ten years is much less than ten times the first-year figure. Enter the total you expect over the period, not a rate.
- **The parabola is typical,** not the part's own curve: the coefficient varies by about ±18% and the turnover by several degrees. Near 25 °C that barely matters; at the extremes it does.
- The MHz temperature figure is taken as a symmetric ± limit, which is how it is specified, even though the real AT-cut curve is not symmetric.
- Pulling is carried in as a signed figure from the crystal load tool and is treated as exact.
- **Tolerance never includes temperature on a crystal datasheet.** The temperature stability is a separate line, often a separate ordering table, and the two add: ±20 ppm tolerance with ±30 ppm over –40/+85 °C is up to ±50 ppm before ageing. A packaged **XO or TCXO** is different: its "frequency stability" is often one all-inclusive figure covering tolerance, temperature, supply and load, sometimes ageing too. Read the footnote; if it is all-inclusive, enter it as Tolerance and leave the rest at zero.

What it deliberately does not do

- **Compensated oscillators.** A TCXO or an RTC with digital trimming cancels most of the temperature term; its datasheet gives a residual stability, which can be entered as Temp. stability on the MHz pill.
- **Short-term stability and jitter.** Phase noise and Allan deviation are a different question from where the average frequency sits.
- **Protocol limits.** Whether a budget meets USB, CAN or Ethernet timing needs those standards' own tolerances and belongs with the tools for each.

How it was checked

By hand, MHz defaults: 20 + 30 + 3 = 53 ppm worst case; $\sqrt{(400 + 900 + 9)} = 36.18$ ppm RSS; 53 ppm of 16 MHz is 848 Hz; 53 × 86.4 ms = 4.579 s a day; 53 × 31.56 s = 27.88 minutes a year. With –25 ppm of pulling the window becomes –78 to +28 ppm and the worst case reads –78, signed.

32.768 kHz at 0 °C: drift $-0.034 \times 25^2 = -21.25$ ppm; with ± 23 ppm of limits the window is -44.25 to $+1.75$ ppm, worst -44.25 ppm, -3.82 s a day. Switching pills resets pulling, which on first build carried over from one family to the other.

PLL multiplication factor

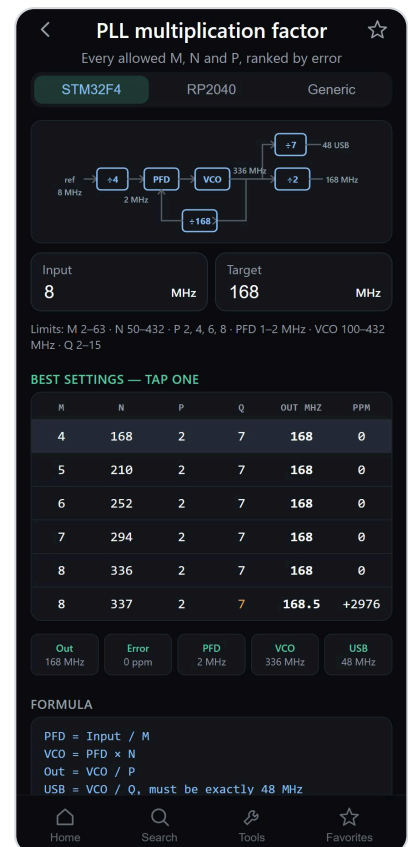
What it computes

Every combination of input divider M, feedback multiplier N and output divider P that a chip's PLL allows, walked exhaustively, with the ones that land closest to a target frequency listed first — each with its error in ppm and the frequencies at the phase detector and the VCO, so the limits can be seen being respected.

Source

The limits are taken from the vendors' own code rather than from summaries, which disagree with each other:

- **STM32F4** — ST's HAL driver ([stm32f4xx_hal_rcc.h](#), [stm32f4xx_hal_rcc_ex.h](#)): PLLM 2–63, PLLN 50–432, PLLP 2, 4, 6 or 8, PLLQ 2–15, and the VCO input between 1 and 2 MHz. The VCO output window of 100–432 MHz and the 168 MHz system clock of the F405/407 are from RM0090.
- **RP2040** — the Pico SDK's own calculator, [vcocalc.py](#): reference divider 1–63 with at least 5 MHz after it, FBDIV 16–320, VCO 750–1600 MHz, and two post-dividers each 1–7. The 133 MHz system clock rating is the datasheet's.



Why the formulas are these

An integer-N PLL has one loop and three dividers:

$$\begin{aligned} \text{PFD} &= \text{Input} / M \\ \text{VCO} &= \text{PFD} \times N \\ \text{Out} &= \text{VCO} / P \end{aligned}$$

The input is divided down to the **phase-frequency detector** (PFD) rate. The VCO runs at N times that, because the loop compares the VCO divided by N against the PFD reference and steers the VCO until they match. The output is the VCO divided down again. Each stage has a window it must stay inside: the PFD because the detector and its filter are designed for a range, the VCO because the oscillator only tunes over a range. That is what makes this a search rather than a division — the obvious M and N often put one of the stages outside its window.

Ranking. Rows are ordered by error first. Among equally good rows:

- on the STM32F4, a row whose VCO also divides by some Q to exactly 48 MHz comes first, since USB OTG will not work on anything else;

- then the **higher PFD**. ST's manual recommends a 2 MHz VCO input to limit jitter, and in general a faster comparison lets the loop correct the VCO more often and filter less, which is quieter;
- then the higher VCO.

That is why 8 MHz to 168 MHz on the STM32F4 comes out as M 4, N 168, P 2, Q 7 (PFD 2 MHz, VCO 336 MHz, USB 48 MHz) ahead of the equally exact M 8, N 336, which runs the detector at 1 MHz.

RP2040's two post-dividers multiply, so the tool offers each distinct product once, as the pair with the larger first divider — the SDK prefers that split for power.

Assumptions and limits

- **Integer-N only.** Many modern PLLs add a fractional part to N, which can hit almost any target but trades it for spurs and depends on each vendor's modulator. Not modelled.
- The preset limits are for the named parts. Other members of a family can differ — the F401, for one, needs its VCO in 192–432 MHz, and the RP2350 is not the RP2040 — so use Generic with the right reference manual when in doubt.
- Lock time, loop-filter design and phase noise are not addressed; the tool answers only which settings are legal and how close they land.
- On Generic, the search stops at three million combinations to stay responsive and says so; narrow the ranges for a complete answer.

What it deliberately does not do

- **Vendor clock trees beyond the PLL** — AHB and APB prescalers, peripheral clock muxes, flash wait states for the chosen frequency. Those are the next steps after this one, and every family arranges them differently.
- **Other presets.** The Generic pill covers any integer-N PLL given its manual; a preset is only worth adding for a part used often enough to save the reading.

How it was checked

Against the configurations the vendors ship: 8 MHz to 168 MHz on the STM32F4 gives M 4, N 168, P 2, Q 7 with USB at exactly 48 MHz, and from a 25 MHz crystal M 25, N 336, P 2, Q 7 — the standard setting for that crystal. 180 MHz is flagged as above the part's 168 MHz rating and as leaving USB at 45 MHz.

On the RP2040, 12 MHz to 125 MHz gives VCO 1500 MHz with post-dividers 6 and 2, and 133 MHz gives VCO 1596 MHz with 6 and 2 — both exactly the Pico SDK's own default settings. The full RP2040 search takes about 3 ms.

ADC resolution / quantization

What it computes

For an ideal N-bit converter and its reference: the size of one step (the LSB), the code a given input produces, the voltage that code stands for, and the quantization error between the two, in volts and in LSB. It works both ways: type a voltage to get the code, or type a code — say, one read off a register — to get the voltage. The range of inputs and codes is shown under the fields.

Two pills cover the two ways converters are built:

- **Single-ended** — 0 to VREF, codes 0 to $2^N - 1$. Every MCU ADC (AVR, STM32, RP2040, ESP32) works this way.
- **Bipolar $\pm$** — $-FSR$ to $+FSR$, codes -2^{N-1} to $2^{N-1} - 1$ in two's complement. Differential converters such as the ADS1115 work this way.

Source

- The ideal transfer function is the one in IEEE Std 1241 and the one MCU datasheets measure their errors against. The ATmega328P datasheet (ADC characteristics) defines offset error as the deviation of the first transition from its ideal place "at 0.5 LSB", gain error against a last transition 1.5 LSB below full scale, and gives ± 0.5 LSB as the ideal absolute accuracy.
- The bipolar pill follows the TI ADS1115 datasheet (SBAS444E): the full-scale range is set by the PGA to ± 6.144 , ± 4.096 , ± 2.048 (the power-up default), ± 1.024 , ± 0.512 or ± 0.256 V; Table 7-1 gives the LSB as $125 \mu\text{V}$ at ± 4.096 V and $62.5 \mu\text{V}$ at ± 2.048 V; output is binary two's complement, clipping at 7FFFh and 8000h.

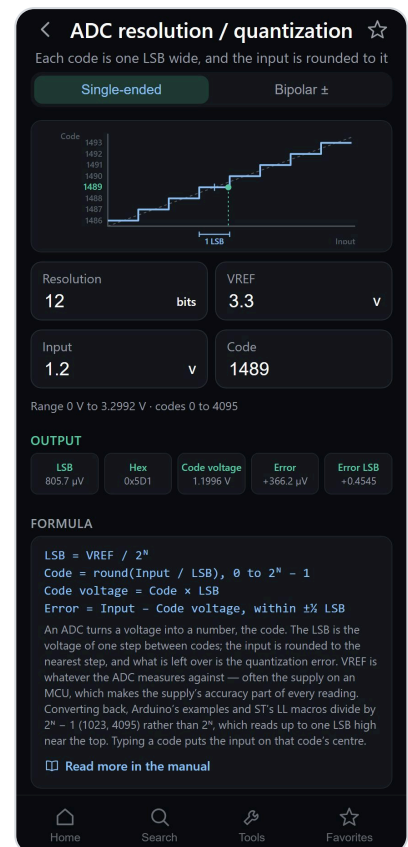
Why the formulas are these

The step. N bits give 2^N codes. Spread over the reference:

Single-ended: $\text{LSB} = \text{VREF} / 2^N$
 Bipolar: $\text{LSB} = 2 \times \text{FSR} / 2^N$

The bipolar span is twice FSR because it runs from $-FSR$ to $+FSR$. The ADS1115 datasheet writes its Equation 4 as $\text{LSB} = \text{FSR} / 2^{16}$, but its own table ($125 \mu\text{V}$ at ± 4.096 V) only works with FSR as the whole 8.192 V span — the tool asks for the $\pm$ figure, as the datasheet's register table quotes it, and doubles it.

The code. Code k stands for $k \times \text{LSB}$ and covers half an LSB either side, so the ideal converter rounds:



$$\text{Code} = \text{round}(\text{Input} / \text{LSB})$$

The first transition, 0 to 1, is at $\frac{1}{2}$ LSB. That is why the quantization error is $\pm\frac{1}{2}$ LSB rather than 0 to -1 LSB: the steps are centred on the ideal straight line, and the error is the sawtooth between the line and the staircase. The drawing shows eight codes around the input at scale, with the dashed ideal line through the step centres.

The top of the range. The highest code is $2^N - 1$, not 2^N , so the largest voltage a code stands for is $V_{\text{REF}} - 1$ LSB: 3.2992 V for 12 bits on 3.3 V. An input at V_{REF} itself reads the top code, a whole LSB short.

Two's complement. A bipolar converter's register holds negative codes as $2^N + \text{code}$, so -1 reads FFFFh and $-\text{FS}$ reads 8000h. The Hex cell shows the register as it would be read, the Code field the signed value it means.

Dividing by $2^N - 1$. Arduino's ReadAnalogVoltage example computes $\text{code} \times 5.0 / 1023$, and ST's LL macro divides by 4095. That treats the top code as exactly V_{REF} , stretching the scale: the reading is $k / (2^N - 1)$ instead of $k / 2^N$ of V_{REF} , high by up to one LSB near the top. Small beside a real ADC's errors, but a systematic one, and it is why a 3.3 V input "reads 3.3 V" on those examples and 3.2992 V here.

Assumptions and limits

- **An ideal converter.** Offset, gain error, INL, DNL and noise are not modelled; on a real MCU ADC they add up to several LSB, far more than the quantization shown here. Their datasheet figures are in LSB, which is what this tool gives the size of.
- **V_{REF} is taken as exact.** On most MCUs it is the supply, so the supply's own tolerance scales every reading. A 1% regulator is 41 LSB at 12 bits.
- **Inputs past the range** read the end code; the tool says so and the error then is the overshoot, not quantization. On the ADS1115 the ± 4.096 and ± 6.144 V settings are scaling only — the pins themselves never go past $V_{\text{DD}} + 0.3$ V or below $\text{GND} - 0.3$ V.
- **Single-ended on a bipolar part.** The ADS1115 measuring one input against ground uses only codes 0000h to 7FFFh, so it is a 15-bit converter in that mode. Model it on the bipolar pill with 16 bits, and read only the positive half.

What it deliberately does not do

- **SNR, ENOB, oversampling.** The noise that quantization adds, and what averaging buys back, is the next tool, SNR estimation.
- **DAC.** The reverse conversion has its own tool.
- **Offset-binary and sign-magnitude codes.** Some bipolar converters output these instead of two's complement; they are rare enough on current parts to leave out.

How it was checked

By hand, single-ended 12 bits on 3.3 V: $\text{LSB} = 3.3 / 4096 = 805.66 \mu\text{V}$; $1.2 \text{ V} / 805.66 \mu\text{V} = 1489.45$, code 1489 = 0x5D1, code voltage 1.19963 V, error $+366.2 \mu\text{V} = +0.4545$ LSB. Code 4095 gives 3.29919 V; 3.4 V reads 4095 and is flagged as over range.

Bipolar 16 bits on ± 2.048 V: LSB $62.5 \mu\text{V}$, matching the ADS1115's Table 7-1. $-1.23456 \text{ V} / 62.5 \mu\text{V} = -19752.96$, code -19753 , register $0xB2D7$ ($65536 - 19753 = 45783$); error $+2.5 \mu\text{V} = +0.04$ LSB. At ± 4.096 V the LSB is $125 \mu\text{V}$, again the datasheet's figure. Code -32768 reads $0x8000$ and -4.096 V, the datasheet's $-\text{FS}$. 24 and 32 bits keep enough figures in the code voltage to tell neighbouring codes apart, and the hex fits its cell.

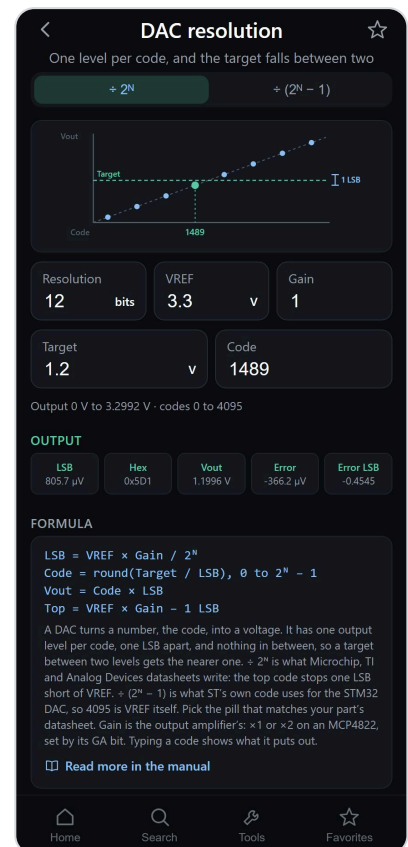
DAC resolution

What it computes

For an N-bit DAC, its reference and its output gain: the step between neighbouring output levels (the LSB), the code to write for a target voltage, the voltage that code really puts out, and how far that is from the target, in volts and in LSB. It works both ways — type the voltage you want, or type a code to see what it outputs. The output range is shown under the fields.

Source

- **Microchip MCP4802/4812/4822** datasheet (DS20002249B):
 $V_{OUT} = V_{REF} \times D / 4096$ with the GA bit at 1 ($\times 1$), and $2 \times V_{REF} \times D / 4096$ with GA at 0 ($\times 2$), from the internal 2.048 V reference; its table gives 0.5 mV per step at $\times 1$ and 1 mV at $\times 2$. The MCP4725 datasheet writes the same $V_{REF} \times D_n / 4096$ with $V_{REF} = V_{DD}$.
- **ST's LL driver** for the STM32 DAC (`stm32f4xx_ll_dac.h`):
`__LL_DAC_DIGITAL_SCALE` is 0xFFF, and
`__LL_DAC_CALC_VOLTAGE_TO_DATA` multiplies by it and divides by V_{REF} , i.e. $code = V \times 4095 / V_{REF}$.



Why the formulas are these

One level per code. A DAC puts out one voltage for each code and nothing in between:

$$V_{out} = Code \times LSB$$

so a target that is not exactly a level gets the nearer one, at most half an LSB away. That is why the picture is points, not a staircase: eight levels around the target at scale, the target a line between two of them, the chosen one in green.

Two divisors, and why it matters. How big the step is depends on what the datasheet divides by:

$\div 2^N$:	$LSB = V_{REF} \times Gain / 2^N$	top code = $V_{REF} \times Gain - 1 \text{ LSB}$
$\div (2^N - 1)$:	$LSB = V_{REF} \times Gain / (2^N - 1)$	top code = $V_{REF} \times Gain$ exactly

Microchip, TI and Analog Devices write 2^N : a resistor string or R-2R ladder of 2^N equal steps, of which the top code uses $2^N - 1$, so the output never quite reaches VREF. ST's own code for the STM32 DAC uses $2^N - 1$, so 4095 is VREF itself. ST's documents do not all agree — a user on ST's community forum points out that RM0444 and AN3126 write 4096 — but the driver says 4095, and that user's bench measurements on a

NUCLEO-G071RB matched 4095 within a millivolt where 4096 was 3–5 mV out. The difference is under one LSB, but it is systematic, and it is always largest near the top of the range.

Gain. Many DACs follow the ladder with an amplifier. On the MCP4822 the GA bit picks $\times 1$ or $\times 2$ from its internal 2.048 V reference, which is how a 2.048 V reference makes a 4.095 V range at 1 mV a step.

Assumptions and limits

- **The output cannot reach the rails.** The MCP4822's amplifier swings from 10 mV to $V_{DD} - 40$ mV, and its accuracy holds only between those. The STM32 DAC with its output buffer on is similarly limited near 0 V and V_{REF+} . The top and bottom few codes on the tool's range are therefore not really available on those parts, and with $\times 2$ gain the range can exceed V_{DD} , which the output will not follow either.
- **An ideal DAC.** Offset, gain error, INL and DNL are not modelled; they are specified in LSB, which is what this tool gives the size of.
- **V_{REF} is taken as exact.** When it is V_{DD} , as on the MCP4725, the supply's tolerance scales every output.

What it deliberately does not do

- **Bipolar DACs** ($\pm V_{REF}$ output with offset-binary or two's-complement codes) are rare on current general-purpose parts; the bipolar pill on the ADC tool covers the arithmetic if needed.
- **PWM as a DAC.** Resolution there is set by the timer and the filter, not a ladder; it is a different calculation.
- **SNR and ENOB** are the next tool.

How it was checked

$\div 2^N$, 12 bits, 3.3 V: LSB 805.66 μ V; 1.2 V gives code 1489 (0x5D1), output 1.19963 V, error -366.2 μ V = -0.4545 LSB. Code 4095 outputs 3.29919 V, one LSB short of V_{REF} .

$\div (2^N - 1)$: LSB $3.3 / 4095 = 805.86$ μ V; code 4095 outputs exactly 3.3 V; 1.2 V gives code 1489, output 1.19993 V.

MCP4822 at $\times 2$ (V_{REF} 2.048 V, gain 2): LSB 1 mV, matching the datasheet's table; 3 V is code 3000 exactly; 5 V is flagged as above the 4.095 V range. 8 bits on the same settings: LSB 16 mV. 24 bits: output shown to enough figures to tell neighbouring codes apart.

SNR estimation

What it computes

Two things, one per pill.

- **Estimate** — the signal-to-noise ratio an ADC can reach from the noises that can be calculated: quantization, the process gain of oversampling, and sampling-clock jitter. Each is shown on its own, then added as noise powers into one SNR and expressed as an effective number of bits (ENOB).
- **ENOB ↔ SINAD** — the conversion between the two figures ADC datasheets quote, both ways, with how many of the converter's bits noise and distortion take away.

The picture is a level diagram in dBFS: full scale at the top, the signal, and each noise floor as a line. The gap from the signal down to the total floor is the SNR, and the highest noise line is the one worth fixing.

Source

Walt Kester's Analog Devices tutorials:

- **MT-001**, "Taking the Mystery out of the Infamous Formula, $\text{SNR} = 6.02N + 1.76 \text{ dB}$ ": the ideal SNR of an N-bit converter for a full-scale sine measured over dc to $f_s/2$, and the correction $10 \log(f_s / 2BW)$ it calls process gain.
- **MT-003**, "Understand SINAD, ENOB, SNR, THD, $\text{THD} + N$, and SFDR": ENOB from SINAD, and its Equation 2, which adds the level below full scale so ENOB is normalised to full scale.
- **MT-007**, "Aperture Time, Aperture Jitter, Aperture Delay Time": the SNR limit set by jitter, and that the total jitter is the root-sum-square of the sampling clock's and the ADC's own aperture jitter.

The ENOB pill's defaults come from the **RP2040 datasheet**, section 4.9.3: SINAD 54.0 dB typical at 997 Hz and almost full scale, ENOB 8.7.

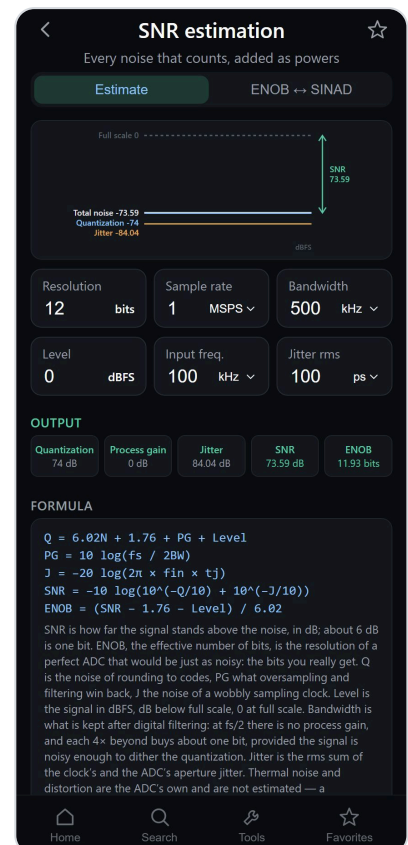
Why the formulas are these

Quantization. Rounding to the nearest code leaves an error spread evenly over $\pm 1/2$ LSB, whose rms value is $q / \sqrt{12}$. A full-scale sine has an rms value of $2^N q / (2\sqrt{2})$. Their ratio, in dB, is

$$\text{SNR} = 6.02N + 1.76 \text{ dB}$$

— 6.02 dB per bit, which is $20 \log 2$, plus a constant from the shape of a sine. 12 bits: 74.0 dB. 16 bits: 98.1 dB.

Level. The quantization noise stays where it is when the signal gets smaller, so the SNR falls one dB for every dB below full scale. A 12-bit ADC reading a signal at -20 dBFS has 54 dB of SNR, the same as an ideal 8.7-bit



one. That is the most common way resolution is wasted.

Process gain. Quantization noise is spread evenly from dc to $f_s/2$. Filter the result digitally down to a bandwidth BW and only the share of the noise inside BW remains:

$$PG = 10 \log(f_s / 2BW)$$

Four times the bandwidth needed is 6.02 dB, one extra bit. MT-001's example: a 65 MSPS ADC with channels 30 kHz wide gains 30.3 dB, 65 dB becoming 95.3 dB.

Jitter. If the instant of sampling wanders by t_j rms, the error it causes is the signal's slope times that wander. For a full-scale sine at f_{in} the result is independent of amplitude:

$$SNR(jitter) = -20 \log(2\pi \times f_{in} \times t_j)$$

It depends on the input frequency, not the sample rate. 100 ps is harmless at 1 kHz (124 dB) and ruinous at 100 MHz (24 dB). MT-007 notes that 14-bit performance at 100 MHz needs under 0.1 ps; the tool gives 84 dB at exactly 0.1 ps, just under the 86 dB of 14 bits.

Adding them. Uncorrelated noises add as powers:

$$SNR = -10 \log(10^{(-Q/10)} + 10^{(-J/10)})$$

so the total is always a little worse than the worse of the two, and a term 10 dB better than the other barely counts. When jitter is the lower one, the tool says so — more bits will not help.

What ENOB means. ENOB, the *effective number of bits*, answers the question "how many bits is this converter really worth?". The resolution on the datasheet, 12 bits say, only counts the codes the ADC can output. A real ADC also adds its own noise and distortion, so the last few bits are buried in them: they change from reading to reading without telling you anything about the signal. ENOB is the resolution of a perfect converter that would be exactly as noisy as the real one. A 12-bit ADC with an ENOB of 8.7 gives, on a changing signal, the same quality as a flawless 8.7-bit ADC; the bottom 3.3 bits are noise. ENOB is not a whole number, because it is worked back from a measured ratio, not counted.

What SINAD means. SINAD, the *signal-to-noise-and-distortion ratio*, is that measured ratio: a pure sine is fed in, and the power of the sine is compared with the power of everything else in the output — noise and the harmonics that distortion adds. SNR is the same measurement leaving the harmonics out, which is why a datasheet's SNR is always the higher of the two. Because a perfect N-bit converter has an SINAD of $6.02N + 1.76$ dB, the two figures are the same fact in different units, and datasheets quote either.

ENOB from the ratio. The SNR equation solved for N, with the level term of MT-003:

$$ENOB = (SNR - 1.76 - Level) / 6.02$$

On the ENOB pill SINAD takes the place of SNR, as the datasheets do. The RP2040's 54.0 dB gives 8.68 bits, and its datasheet says 8.7: of its 12 bits, 3.3 are lost to noise and distortion. Its SNR is 61.5 dB but its THD is

–55 dB — its INL and DNL errors (the DNL spikes are erratum RP2040-E11) cost more than the noise.

Assumptions and limits

- **Oversampling needs noise.** Process gain assumes the quantization error is random. A clean, slow signal on a quiet ADC can give the same code every time, and averaging a constant gives nothing. MT-001 warns about this correlation; the cure is noise or dither of about an LSB at the input.
- **Thermal noise and distortion are not estimated.** They are the ADC's own and only its datasheet knows them. The Estimate pill gives the ceiling the architecture allows; the ENOB pill gives what the part actually does.
- **No noise shaping.** Delta-sigma converters push quantization noise out of band and gain far more than 10 log per octave of oversampling. Their datasheets give SNR directly.
- **SINAD at other levels.** MT-003's level correction assumes the noise does not change with level. Distortion usually drops at lower levels, so ENOB normalised from a –6 dBFS measurement can read better than at full scale.

What it deliberately does not do

- **THD, SFDR, THD + N.** Read from the datasheet, not calculated.
- **Noise-free resolution and effective resolution** from rms input noise, the measure for dc and slow signals. MT-003 warns not to confuse them with ENOB; they are a different calculation from different data.

How it was checked

Default Estimate: 12 bits, 1 MSPS, 500 kHz, 0 dBFS, 100 kHz, 100 ps. $Q = 6.02 \times 12 + 1.76 = 74.0$ dB; $J = -20 \log(2\pi \times 10^5 \times 10^{-10}) = 84.04$ dB; total 73.59 dB; ENOB 11.93 bits.

MT-001's process-gain example: 65 MSPS and 30 kHz give 30.35 dB. MT-007's jitter example: 14 bits, 100 MHz, 0.1 ps give $J = 84.04$ dB, total 81.91 dB, with the jitter warning. 40 MHz of bandwidth at 65 MSPS is refused as past $f_s/2$. With no jitter at –1 dBFS, 14 bits give 85.04 dB and an ENOB of 14.0, normalised.

ENOB pill: SINAD 54.0 dB at 0 dBFS gives 8.678 bits — the RP2040's 8.7 — with 3.32 bits lost; ENOB 10 gives SINAD 61.96 dB.